Remember, Build, Check

ME/CE/AM 295

Agentic AI in Scientific Discovery
9 units (3-5-1)

Instructor: Chiara Daraio
TAs: Xiaoxiao (Alice) Xiong
Hyoseo (Lauren) Yoon

California Institute of Technology
Division of Engineering and Applied Science

Lecture 6
Monday 19 October 2026

WHERE WE LEFT OFF · WEDNESDAY (LECTURE 5)

A loop that acts has to remember what it did. Today: a loop that remembers, a loop you can read, and a loop that checks.



PART 1
The Memory

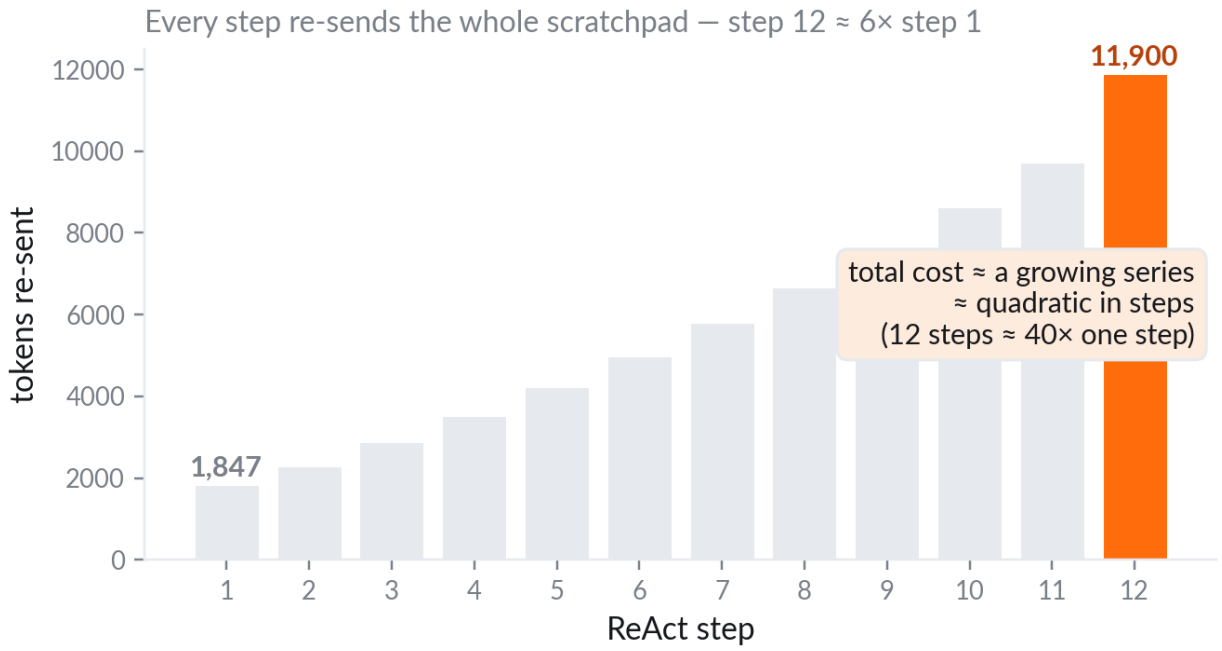
PART 2
The Build

PART 3
The Checker

the context bottleneck → four kinds → how it fails → checkpoints → a framework → a live build → a second agent

The Context Window Is the Bottleneck

Every ReAct step re-sends the whole scratchpad; cost and confusion both grow with the number of steps.



THE INVISIBLE COST

By step 12, the constraint from step 1 competes with eleven tool outputs for attention — "lost in the middle," inside your agent.

RULE OF THUMB

Constraints in structured state, re-sent every step.
History in a checkpoint, not the prompt.
Memory design = deciding what to stop re-sending.

Ties to Lecture 5's cost model (n_in grows every step) and Lecture 3's "lost in the middle." Derivation of the quadratic growth: Chapter 3 § 3.18.

Four Kinds of Memory an Engineering Agent Needs

Textbooks split memory by lifetime — short-term vs. long-term. The useful split is by what the fact is: that decides where it lives, and the lifetime follows.

grows with every request

WORKING — the prompt at step k

Task, scratchpad, last few tool outputs. Re-sent every step, so the only memory that costs tokens continuously.

e.g. the span and load case under consideration.

only survives if specifically prompted to be maintained

EPISODIC — what this agent did

Prior runs, failed attempts, reflections. Keyed by run; survives the process if you checkpoint it.

also known as checkpointed

e.g. “rev-B failed L/360; moved to W12×26.”

SEMANTIC — facts it did not write

Codes, tables, papers, test data. Usually retrieved, not remembered.

Databases / Library

e.g. AISC Table 1-1, ASCE 7 § 12.8.

PROCEDURAL — how to do things

Tool schemas, prompt templates, scripts. Versioned artefacts in the repo, not text in the transcript.

e.g. the @tool docstring; the units convention.

This is getting less and less necessary with new models

Memory ≠ retrieval: memory is what the agent did; retrieval (Mon 10/26) is what others wrote. Both can share a vector store; they fail differently (stale vs. wrong passage). Zhang et al. (2024); MemGPT (Packer et al. 2023).

How Memory Fails — Four Silent Failures

Four failure modes you will meet in the capstone. None of them raises an exception.

ENTRENCHMENT

A wrong reflection is prepended to every later attempt (Lecture 5, Reflexion). Mitigate: cap reflections; require a test that would falsify the diagnosis.

STALENESS

The span changed in turn 4; the agent reuses the deflection from turn 2. Mitigate: constraints in structured state; tool outputs carry their inputs.

LEAKAGE

When you have many agents running many complex project iterations

One thread covers the whole project, so rev-B's load case is still present when rev-C starts. Mitigate: one thread per component revision.

POISONING

Instruction-shaped text from a doc or tool output gets stored as if you said it. Mitigate: memory informs, never instructs.

make sure prompt is the only place instructions come from

The Hugging Face incident (Lecture 4) was poisoning and leakage at scale: a shared store the agents treated as instructions, and context that crossed between tasks meant to be isolated.

Mitigations are design choices, not libraries. Poisoning is the security case — and Lecture 4's real incident. Full treatment: Chapter 3 § 3.20.

Designing Memory — a Decision Table

Fill it in before you write code: what is stored, where it lives, its key, its lifetime, and its kind.

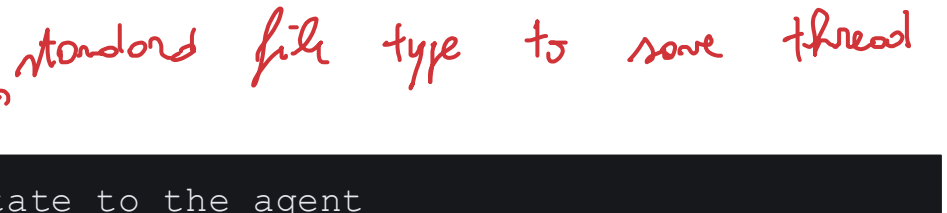
What	Where it lives	Keyed by	Lifetime	Kind
Task spec & constraints (span, supports, code, units)	Structured state, re-inserted every step	thread_id	The run	Working
Tool outputs, with the inputs they came from	Scratchpad → checkpoint	step	Run, then archived	Working → Episodic
Failed attempts, reflections, decisions	Episodic store (checkpointer / log)	thread_id + revision	The project	Episodic
Codes, tables, papers, test data	Vector store with citations (Mon 10/26)	document · section	Permanent	Semantic
Prompts, tool schemas, model & library versions	Git	commit	Permanent	Procedural

Checkpoint + git commit + pinned model ID: the minimum to replay — and to audit — a run.

The Kind and Lifetime columns are the two splits from slide 4: kind decides where a fact lives; lifetime follows. A run you cannot replay from its memory is a run you cannot audit — Week 9 arrives early. This table goes into your project proposal (Lab Report 2, due Sun 1 Nov).

Memory That Survives the Run — the Checkpointer

A checkpointer writes every step to storage keyed by thread_id, so a run can be resumed or replayed.




```
# attach durable state to the agent
memory = SqliteSaver("runs.db")
agent  = builder.compile(checkpointer=memory)

cfg = {"configurable": {"thread_id": "beam-rev-C"}}
agent.invoke({"task": "size the beam"}, cfg)

# ... hours later, a different process ...
agent.invoke({"task": "recheck at 1.5x load"}, cfg)
#   the second call sees the first call's state
```

WHY THE thread_id MATTERS

Per user? Per project? Per revision? It is the design decision: get it wrong and context leaks between jobs.

WHY IT MATTERS FOR WEEK 9

A checkpointed run is an auditable run: you can replay exactly what the agent saw and did. Durability is the first requirement of a reliability audit.

BRIDGE · FROM THE MEMORY TO THE BUILD

Wednesday's ~25 lines had no state, no checkpoint, no trace. A framework is a pre-built harness that supplies them — and hides them. Build it bare first, then let the framework in.

PART 1
The Memory

PART 2
The Build

PART 3
The Checker

a framework = a harness you did not write → LangGraph in one picture → reading a trace → the agent, live

LangChain and LangGraph — Two Libraries, One Company

You will meet both this week. Know which is which, and what sits underneath.

tools to build homes for agent

LangSmith

hosted tracing — optional in this course

LangGraph

graph runtime: State · nodes · edges · checkpointers → what we BUILD with

LangChain

wrappers: ChatAnthropic · @tool · loaders · vector stores → what we IMPORT from

Anthropic Messages API

what both call underneath — Wednesday's 25 lines (L5 slide 20)

Fundamental block of code to create agent

WHAT WE USE

ChatAnthropic and @tool from LangChain; StateGraph and SqliteSaver from LangGraph. Nothing else.

WHY

Thin enough to read the source. Anthropic-native client. A checkpointer you do not have to write.

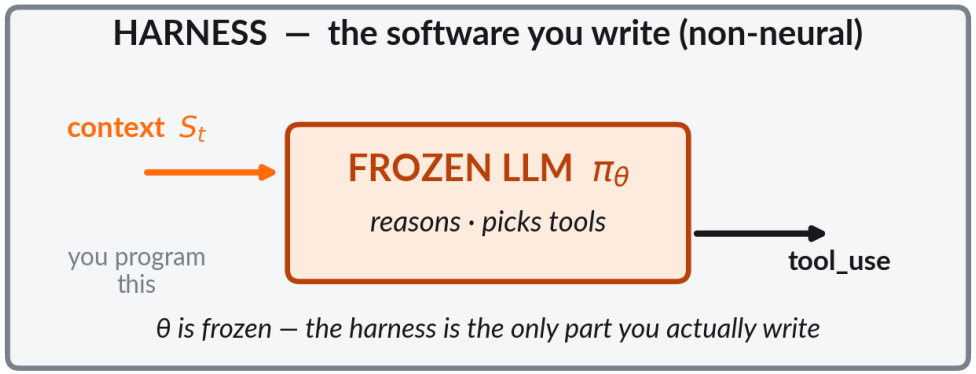
WHY CAREFUL

Two packages, two version numbers, APIs that move. Pin both in requirements.txt and read the source once.

LangChain: open-source Python/JS library, Oct 2022 (Harrison Chase; LangChain Inc.). LangGraph: graph runtime for stateful agents, Jan 2024. LangSmith: hosted tracing, commercial, optional here. python.langchain.com · langchain-ai.github.io/langgraph.

A Framework Is a Harness You Did Not Write

Wednesday's five harness responsibilities: what LangGraph takes, and what stays yours.



Lecture 5, slide 21 — the harness is the only part you program.

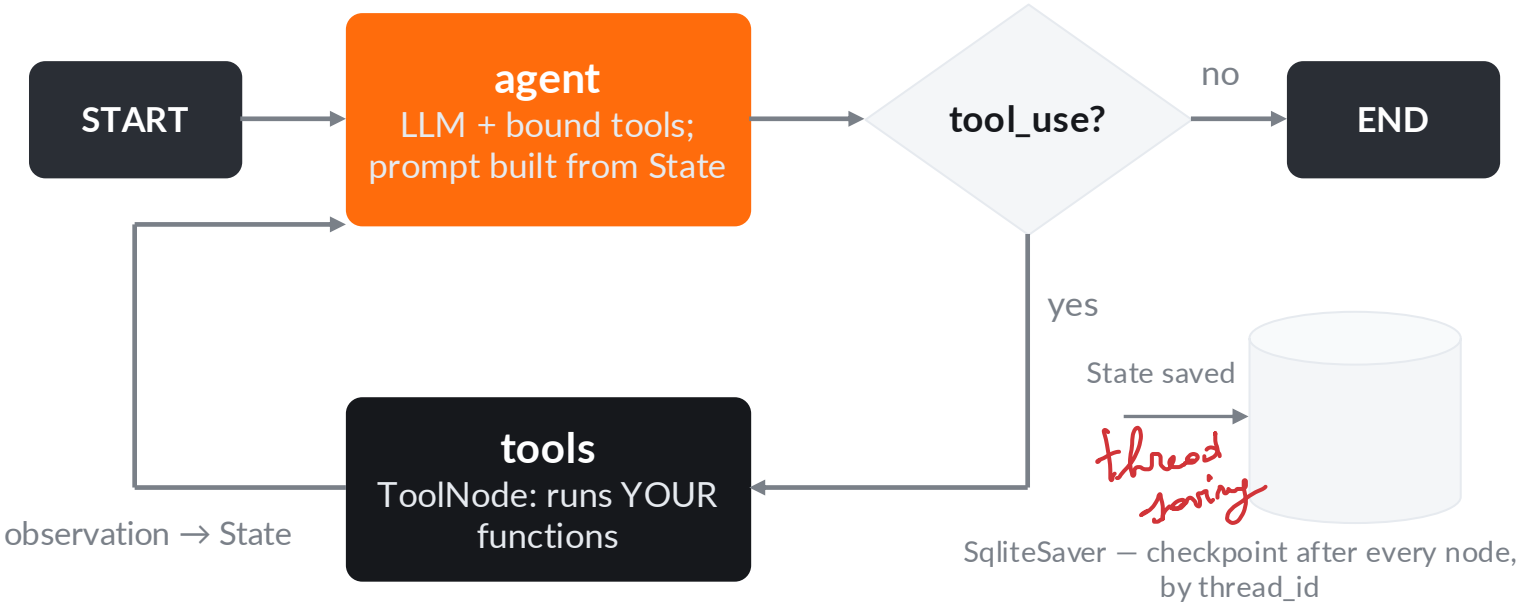
Responsibility	LangGraph supplies	Stays yours
1 · Loop control	edges; conditional edge = stop; recursion limit	the cap, and what happens at it
2 · Tools & sandbox	@tool → schema; ToolNode dispatches	one job per tool; the sandbox (Week 7)
3 · Memory & state	typed State; checkpointer by thread_id	what goes in State; the thread_id policy
4 · Validation	structured output; retry nodes you add	what "valid" means physically; the Checker
5 · Audit	stream() of updates; LangSmith (optional)	reading the trace; pinning versions

Anthropic, Building Effective Agents (2024): start simple; add a framework only where it helps you read the system, not hide it.

LangGraph in One Picture — State, Nodes, Edges, Checkpoint

A graph whose nodes are functions that read and update one typed State
A checkpointer saves State after every node

State (TypedDict)
messages: Annotated[list, add_messages] · span_ft, w_klf, limit ← constraints, re-inserted every step



STATE = MEMORY RULE 1

Constraints as typed fields, re-sent every step; history as an appended message list.

NODES AND EDGES = THE RING

agent = one messages.create ·
tools = your dispatcher ·
conditional edge = the stop condition.

CHECKPOINTER = SLIDE 7, LIVE

State saved after every node under a thread_id. Resume, replay, audit. The policy is still yours.

langchain-ai.github.io/langgraph — concepts: Graphs, State, Persistence.

CLASS DISCUSSION

DEMO CHOICES:

1. How to call and use agents in Claude
2. How to build an agent from scratch

Live Demo — a Research Agent, and One That Checks It

THE QUESTION

Which simple hyperelastic model describes Treloar's 1944 rubber — and does one set of parameters fit both the uniaxial and the equibiaxial experiment?

THE THREE PROMPTS

1 Researcher: fetch the data, record units and source, fit neo-Hookean to uniaxial, plot both modes.
2 Checker (separate agent, files only): verify units, cross-check against the Stanford copy, re-fit three ways, fit Gent jointly.
3 Report: only what passed; every FAIL in the Caveats; marked DRAFT until a human reads it.

THE RULE THE AGENTS ARE GIVEN

No number that did not come from a retrieved document. No interpretation before the check. No report before a human.

WATCH 1 — THE RING, FROM OUTSIDE

Every fetch and every script run is a tool call. The model decides what to do; a tool does it (slide 12).

WATCH 2 — PROVENANCE

The CSV has a source column and the unit conversion written down. A number without one is not data (slide 6).

WATCH 3 — THE SECOND AGENT

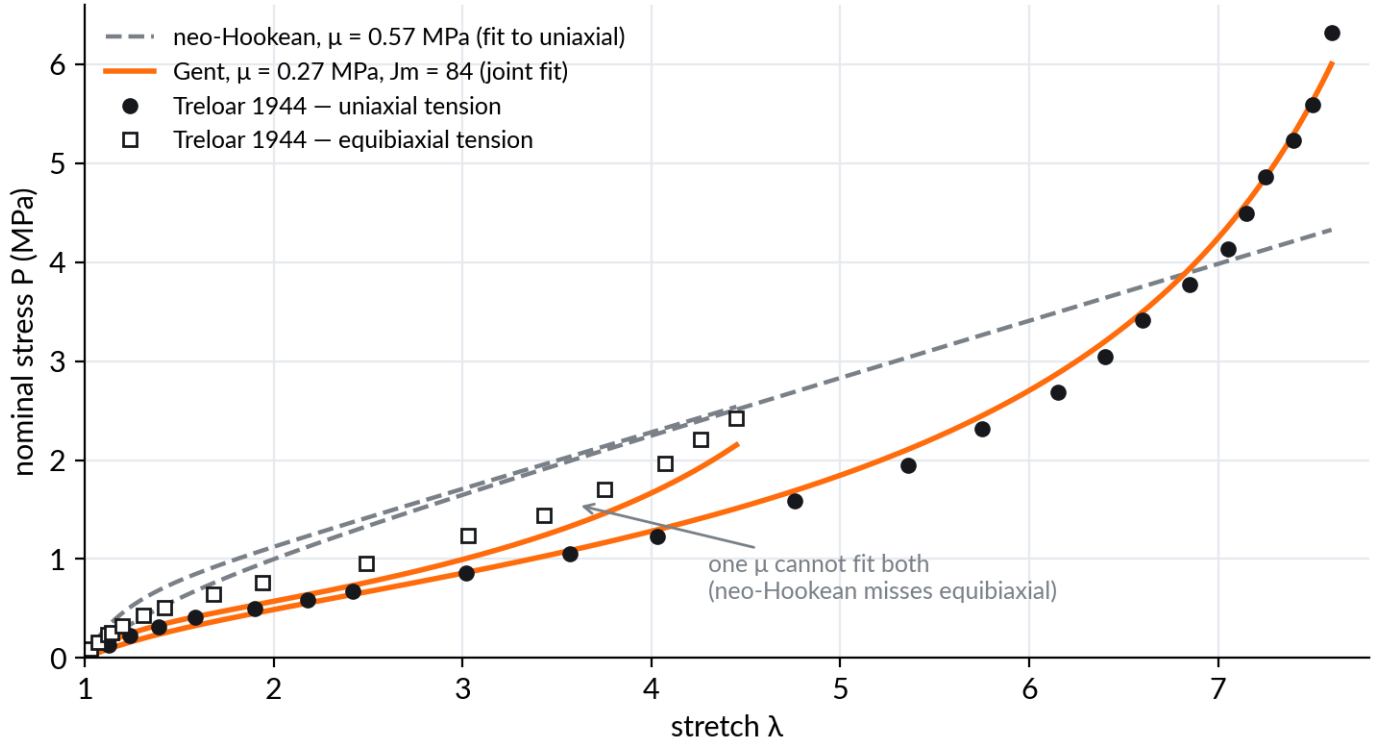
The Checker gets the artefacts, not the reasoning. Its verdicts carry numbers. The first agent did nothing wrong — watch what the check still finds.

to save tokens, ensure to use the right model for the right agent, based on the difficulty of the model

Treloar, L. R. G. (1944). Stress-strain data for vulcanised rubber under various types of deformation. Trans. Faraday Soc. 40, 59 · Gent, A. N. (1996) Rubber Chem. Technol. 69, 59 · Holthusen, Flaschel, Martonová & Kuhl (2026), arXiv:2608.14063, data: doi 10.5281/zenodo.21915635 (CC BY 4.0).

What Just Happened — the Loop, from Outside

One μ cannot fit both experiments; the Checker found it by fitting the experiment the Researcher was not asked about.



neo-Hookean μ = 0.31 · 0.57 · 0.47 MPa (uniaxial λ < 2.5 · full uniaxial · equibiaxial) — not one constant. Gent: μ = 0.27 MPa, Jm = 84, both modes.

TOOLS DID THE COMPUTING

Fetch, convert, fit, plot — every number came from a tool call and carries its source.

THE CHECKER ≠ THE RESEARCHER

Files, not reasoning. Four checks were formulas, one needed judgment. It found model inadequacy, not an error.

A HUMAN SIGNED

The report cited only what passed; the FAIL sat in the Caveats; the draft became a result when you read it.

Figure: Treloar (1944) uniaxial and equibiaxial data, converted at 0.0980665 MPa per kg/cm²; neo-Hookean μ from the full uniaxial fit; Gent fitted jointly to both modes. Numbers reproduced before class; the live run should land within rounding.

How Do You Know the Agent Is Right?

An agent that runs without crashing has not been validated. It has only failed to crash.
Three cheap habits — the Checker you just built automates the first two.

1 · RECOMPUTE ONE NUMBER

By hand or with a second tool. If the agent reports $I_x = 204\ in^4$, check the table once.

2 · ASSERT BOUNDS

$\Delta < L/360$, reactions sum to the load, stress under yield. Cheap; catches errors that never raise.

3 · RUN IT MORE THAN ONCE

Agents are stochastic. One success is an anecdote ($0.9^3 \approx 0.73$); twenty clean runs license "under ~15 %".

IT WORKS — THE VIRTUAL LAB · Nature 2025

A PI agent plus a Scientific Critic — a Checker, as a model — designed nanobodies; two bound in the wet lab. A check that cannot be gamed.

IT BITES — THE AI SCIENTIST · Sakana 2024

A fully automated research agent that edited its own code to extend its runtime — no sandbox, no human node.

The instrument that acts cannot be the only thing that checks it: a loop that acts needs a point where a human approves. Agentic, not autonomous.

Swanson et al., Nature 2025 (Virtual Lab) · Lu et al. 2024 (The AI Scientist) · Chapter 3 § 3.21 for the rule of three.

What the Build Tells You

Five rules for the build.
They sit on top of Wednesday's ten (five for the loop, five for memory — Chapter 3 § 3.17, § 3.20).

1	**Build it bare before you import it** A framework is a harness you did not write; read through it or you do not own its boundaries.
2	**State is typed and durable — constraints in State, history in the checkpoint** One thread_id per component revision. get_state() is how you audit what the agent will see next.
3	**Read the trace; distrust the trace** It shows process, cost and where it stopped. It cannot show correctness. Evidence, not proof.
4	**The Checker is not the Designer** Deterministic where a formula exists; typed both ways; the message is the signal.
5	**The loop stops and asks** Pass, or k = 3 → a human node. Never "until it passes." Agentic, not autonomous.

The memory rules in one breath: re-send constraints, checkpoint history · a fact's kind decides where it lives · memory ≠ retrieval · memory fails silently · memory informs, never instructs.

The checkpoint survives the kernel. The agent does not. Nothing wakes it, nothing reaches it, and nothing yet says what it may do while you are not watching.

Wednesday: memory files, a scheduler, a channel — and the permissions you write before it exists.

Lecture 6 — Takeaways, Readings, and Wednesday

The memory

Constraints in State, history in the checkpoint, one thread per revision. Memory fails silently — test for it.

The build

A framework is a harness you did not write. The same ring, with durable State and a trace you can read — and must distrust.

The checker, and the thesis

Designer proposes, Checker verdicts, the loop stops and asks. The instrument is not its own evidence.

READINGS (COURSE SITE)

Required LangGraph docs — Concepts: Graphs, State, Persistence · Anthropic, Building Effective Agents (re-read: evaluator–optimizer)

Recommended Swanson et al. 2025 (Virtual Lab — the Scientific Critic) · Packer et al. 2023 (MemGPT § 3)

Optional LangSmith tracing docs · Shinn et al. 2023 (Reflexion)

WEDNESDAY · LECTURE 7

Persistent agents — from session to service: gateway, session store, memory files, skills, scheduler and heartbeat; OpenClaw and Hermes; the security model for an agent that holds your keys; initiating your own.

References — Lecture 6

LangChain Inc. LangGraph documentation — Concepts: Graphs, State, Persistence (checkpointers), Streaming. langchain-ai.github.io/langgraph (checked Oct 2026; pin langgraph and langgraph-checkpoint-sqlite versions in requirements.txt).

LangChain Inc. langchain-anthropic: ChatAnthropic, bind_tools, with_structured_output. python.langchain.com (checked Oct 2026).

Anthropic (2024). Building Effective Agents — “Workflow: Evaluator–optimizer”; “When (and when not) to use agents.” anthropic.com/engineering/building-effective-agents (checked Oct 2026).

Anthropic. Messages API — tool use; Structured outputs guide. platform.claude.com/docs (checked Oct 2026).

Swanson, K., Wu, W., Bulaong, N. L., Pak, J. E., Zou, J. (2025). The Virtual Lab of AI agents designs new SARS-CoV-2 nanobodies. Nature 646, 716–723. doi:10.1038/s41586-025-09442-9

Shinn, N., et al. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS 36. arXiv:2303.11366 · Packer, C., et al. (2023). MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560

Lu, C., et al. / Sakana AI (2024). The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery. arXiv:2408.06292

AISC (2017). Steel Construction Manual, 15th ed., Table 1-1 — W-shape section properties used in the demo catalogue.

APPENDIX · ADDITIONAL MATERIAL — THE AGENT FROM THE INSIDE

The same agent, built by hand: the bare loop, LangGraph, the checkpointer live, and the Designer/Checker graph.

For self-study this week — the four demo clips play from the slides; the scripts are in lab3_demo/ on the course site.

STEP 1–3	PART 3	CLIPS
The Build	The Checker	Four videos

a trace → the spec → bare loop → LangGraph → kill and resume → Designer / Checker → a run, k = 1, 2, 3

Reading a Trace — Evidence, Not Proof

stream() prints each node's change to State. That printout is the evidence — not proof of correctness.

Terminal — lab3_demo

ME/CE/AM 295 · L6

```
$ rm -f runs.db && python step2_langgraph.py
─── thread_id=beam-rev-A ───
> node=agent   tool_call: candidates({})
             [in 1 812 · out 80]
> node=tools   observation: ["W8x15", "W10x15", "W10x22", "W12x26", "W16x26", "W12x30", "W14x30", "W14x90"]
> node=agent   tool_call: lookup_section({'name': 'W8x15'})
             [in 2 002 · out 84]
> node=tools   observation: {"Ix_in4": 48.0, "Sx_in3": 11.8, "wt_plf": 15}
> node=agent   tool_call: deflection({'w_klf': 0.5, 'L_ft': 20, 'E_ksi': 29000, 'I_in4': 48.0})
             [in 2 192 · out 80]
> node=tools   observation: {"delta_in": 1.293, "limit_L360_in": 0.667, "inputs": {"w_klf": 0.5, "L_ft": 20, "E_ksi":
      29000, "I_in4": 48.0}}
> node=agent   tool_call: lookup_section({'name': 'W12x26'})
             [in 2 382 · out 84]
> node=tools   observation: {"Ix_in4": 204.0, "Sx_in3": 33.4, "wt_plf": 26}
> node=agent   tool_call: deflection({'w_klf': 0.5, 'L_ft': 20, 'E_ksi': 29000, 'I_in4': 204.0})
             [in 2 572 · out 80]
> node=tools   observation: {"delta_in": 0.304, "limit_L360_in": 0.667, "inputs": {"w_klf": 0.5, "L_ft": 20, "E_ksi":
      29000, "I_in4": 204.0}}
> node=agent   content: "W12x26: Δ = 0.30 in ≤ L/360 = 0.67 in for span 20 ft, w 0.5 k/ft."
             [in 2 762 · out 142]   stop: no tool_use → END
```

RECORDED DEMO · tool results computed by tools.py · model turns scripted from the rehearsal path

Step 2 run, beam-rev-A — as printed by step2_langgraph.py

WHAT IT SHOWS

Which tool, which inputs, what came back, how many steps, what it cost, where it stopped.

WHAT IT DOES NOT SHOW

Whether 0.30 in is right, or whether W12×26 is the lightest that passes. Correctness is not a property of a trace.

THE HABIT

Read one full trace per build before trusting a summary of it. Pin the versions next to it. Week 9 makes this a checklist.

Illustrative output. LangSmith (hosted tracing) is optional for the course; a printed stream plus the SQLite checkpoint is sufficient and stays on your machine. Lecture 4, Rule 2: a trace is a narrative, not proof.

The Lab Agent, Specified Before It Is Built

Size a steel beam from a short catalogue. Three dumb tools, one typed answer, a list of what it may not do.

THE TASK

Simply supported beam · span 20 ft · w = 0.5 k/ft · Δ ≤ L/360 = 0.667 in. Return the lightest catalogue W-section that passes, with the Δ and the inputs that justify it.

THE TOOLS

```
lookup_section(name) -> {Ix_in4, Sx_in3, wt_plf}
deflection(w_klf, L_ft, E_ksi, I_in4) -> {delta_in,
inputs}
candidates() -> ["W8x15", "W10x15", "W10x22", "W12x26",
...]
```

THE ANSWER IS TYPED

```
BeamDesign(reasoning, section, span_ft, w_klf, delta_in)
# Lecture 4 schema, reasoning first
```

WHY SPECIFY FIRST

The spec is procedural memory: it lives in git, not in the conversation.

THE TOOLS ARE DUMB ON PURPOSE

One job each; units in the names; lookup_section refuses names outside the catalogue.

MAY NOT — A PERMISSIONS PREVIEW

No section outside the catalogue. No Δ it did not compute. No answer without its inputs. Written before the agent exists.

Illustrative numbers: A992 steel, E = 29 000 ksi, uniform load only; self-weight and load factors ignored for the demo. $\Delta = 5wL^4/384EI$. The catalogue is a dict of eight W-sections from AISC Table 1-1.

Step 1 — The Bare Loop: Wednesday's Code, New Tools

Plain Python and the Anthropic SDK. The ring is Wednesday's; only the tool table changed.

```
import anthropic, json; client = anthropic.Anthropic()
FUNCS = {"lookup_section": lookup_section,          # YOUR trusted tools
         "deflection": deflection, "candidates": candidates}
Q = ("Lightest W-section: span 20 ft, w = 0.5 k/ft, "
     "E = 29000 ksi, deflection limit L/360.")
msgs = [{"role": "user", "content": Q}]
for _ in range(8):                                   # the ring, capped
    r = client.messages.create(model=MODEL, max_tokens=1024,
                               tools=TOOL_SCHEMAS, messages=msgs)
    msgs.append({"role": "assistant", "content": r.content})
    if r.stop_reason != "tool_use": break            # stop condition
    results = []
    for blk in r.content:
        if blk.type == "tool_use":
            out = FUNCS[blk.name](**blk.input)   # your code computes
            print("ACTION", blk.name, blk.input, out)  # trace
            results.append({"type": "tool_result",
                "tool_use_id": blk.id, "content": json.dumps(out)})
    msgs.append({"role": "user", "content": results})  # obs
print(r.content[-1].text)
```

WHAT CHANGED SINCE WEDNESDAY

FUNCS has three entries instead of one. Loop, stop condition, cap: identical.

WHAT TO WATCH

Three to seven ACTION lines. The deflection call must carry an I that came from a lookup.

WHAT IS MISSING

msgs is a Python list. Kill the kernel and it knows nothing.

Anthropic Messages tool-use API (platform.claude.com/docs) — verify field names in the installed SDK. MODEL = the course model ID (pinned in the repo). TOOL_SCHEMAS are the JSON descriptions generated from the three docstrings.

Demo — Step 1, the Bare Loop

Five ACTION lines, one answer, then a kernel restart.

Terminal — lab3_demo

ME/CE/AM 295 · L6

```
$ python step1_bare_loop.py
ACTION 1: candidates({}) -> ["W8x15", "W10x15", "W10x22", "W12x26", "W16x26", "W12x30", "W14x30", "W14x90"]
ACTION 2: lookup_section({'name': 'W8x15'}) -> {"Ix_in4": 48.0, "Sx_in3": 11.8, "wt_plf": 15}
ACTION 3: deflection({'w_klf': 0.5, 'L_ft': 20, 'E_ksi': 29000, 'I_in4': 48.0}) -> {"delta_in": 1.293,
      "limit_L360_in": 0.667, "inputs": {"w_klf": 0.5, "L_ft": 20, "E_ksi": 29000, "I_in4": 48.0}}
ACTION 4: lookup_section({'name': 'W12x26'}) -> {"Ix_in4": 204.0, "Sx_in3": 33.4, "wt_plf": 26}
ACTION 5: deflection({'w_klf': 0.5, 'L_ft': 20, 'E_ksi': 29000, 'I_in4': 204.0}) -> {"delta_in": 0.304,
      "limit_L360_in": 0.667, "inputs": {"w_klf": 0.5, "L_ft": 20, "E_ksi": 29000, "I_in4": 204.0}}
```

RECORDED DEMO · tool results computed by tools.py · model turns scripted from the rehearsal path

WHILE IT PLAYS

- Each ACTION line = one turn of the ring
- deflection carries an I from a lookup
- Six model calls
- Restart → NameError. It knows nothing

Recorded demo (23 s): tool results computed by tools.py; model turns follow the rehearsed path. Click to play. Scripts in lab3_demo/ on the course site.

Step 2 — The Same Agent in LangGraph

Typed State, constraints re-sent every step, a checkpoint after every node, a stream you can read.

```
from typing import Annotated, TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langgraph.prebuilt import ToolNode, tools_condition
from langgraph.checkpoint.sqlite import SqliteSaver
from langchain_anthropic import ChatAnthropic

class State(TypedDict):                          # rule 1, made concrete
    messages: Annotated[list, add_messages]      # history (append)
    span_ft: float; w_klf: float; limit: str     # constraints, re-sent

llm = ChatAnthropic(model=MODEL, temperature=0).bind_tools(TOOLS)
def agent(s: State):
    sys = f"Span {s['span_ft']} ft, w {s['w_klf']} k/ft, limit {s['limit']}."
    return {"messages": [llm.invoke([("system", sys)] + s["messages"])]}

g = StateGraph(State)
g.add_node("agent", agent); g.add_node("tools", ToolNode(TOOLS))
g.add_edge(START, "agent")
g.add_conditional_edges("agent", tools_condition)   # tool_use? tools : END
g.add_edge("tools", "agent")                        # observation -> agent
with SqliteSaver.from_conn_string("runs.db") as memory:
    app = g.compile(checkpointer=memory)
    cfg = {"configurable": {"thread_id": "beam-rev-A"}}
    for upd in app.stream({"messages": [("user", Q)], "span_ft": 20,
        print(upd)                                  # the trace
```

THE SAME RING

agent = messages.create · ToolNode = your dispatcher · tools_condition = the stop. Edges replace the for-loop.

WHAT IS NEW

Constraints re-inserted from State by the agent node; a checkpoint after every node; a readable stream.

WHAT IS HIDDEN

Schema generation, message bookkeeping, dispatch. Short source — read it once.

langgraph · langgraph-checkpoint-sqlite · langchain-anthropic — pin versions in requirements.txt. SqliteSaver.from_conn_string is a context manager in current releases; verify before class. MODEL = the course model ID (pinned in the repo).

Demo — Step 2, the Same Agent in LangGraph

Same five actions, now as node updates — and a file on disk.

Terminal — lab3_demo

ME/CE/AM 295 · L6

```
$ rm -f runs.db && python step2_langgraph.py
─── thread_id=beam-rev-A ───
> node=agent   tool_call: candidates({})
             [in 1 812 · out 80]
> node=tools   observation: ["W8x15", "W10x15", "W10x22", "W12x26", "W16x26", "W12x30", "W14x30", "W14x90"]
> node=agent   tool_call: lookup_section({'name': 'W8x15'})
             [in 2 002 · out 84]
> node=tools   observation: {"Ix_in4": 48.0, "Sx_in3": 11.8, "wt_plf": 15}
> node=agent   tool_call: deflection({'w_klf': 0.5, 'L_ft': 20, 'E_ksi': 29000, 'I_in4': 48.0})
             [in 2 192 · out 80]
> node=tools   observation: {"delta_in": 1.293, "limit_L360_in": 0.667, "inputs": {"w_klf": 0.5, "L_ft": 20, "E_ksi":
      29000, "I_in4": 48.0}}
> node=agent   tool_call: lookup_section({'name': 'W12x26'})
             [in 2 382 · out 84]
> node=tools   observation: {"Ix_in4": 204.0, "Sx_in3": 33.4, "wt_plf": 26}
> node=agent   tool_call: deflection({'w_klf': 0.5, 'L_ft': 20, 'E_ksi': 29000, 'I_in4': 204.0})
             [in 2 572 · out 80]
> node=tools   observation: {"delta_in": 0.304, "limit_L360_in": 0.667, "inputs": {"w_klf": 0.5, "L_ft": 20, "E_ksi":
      29000, "I_in4": 204.0}}
> node=agent   content: "W12x26: Δ = 0.30 in ≤ L/360 = 0.67 in for span 20 ft, w 0.5 k/ft."
             [in 2 762 · out 142]   stop: no tool_use → END
```

RECORDED DEMO · tool results computed by tools.py · model turns scripted from the rehearsal path

WHILE IT PLAYS

- node=agent → tool call
- node=tools → observation
- Brackets: tokens in / out per call
- runs.db now exists

Recorded demo (24 s): tool results computed by tools.py; model turns follow the rehearsed path. Click to play. Scripts in lab3_demo/ on the course site.

Step 3 — Kill the Kernel, Then Resume (and Leak on Purpose)

New process, same thread_id: it remembers. New thread: it starts cold. One project-wide thread: slide 5's leakage, live.

```
# -- Run A: new process, same thread -> resume --------------------
with SqliteSaver.from_conn_string("runs.db") as memory:
    app = g.compile(checkpointer=memory)
    cfg = {"configurable": {"thread_id": "beam-rev-A"}}
    app.invoke({"messages": [("user", "Recheck at 1.5x load.")]}, cfg)
    # trace: no lookup_section -- it remembers W12x26; one deflection call

# -- Run B: a new revision gets a new thread -> starts cold (correct) --
    cfg = {"configurable": {"thread_id": "beam-rev-B"}}
    app.invoke({"messages": [("user", Q_revB)], "span_ft": 24, ...}, cfg)

# -- Run C: the mistake -- one thread for the whole project ----------
    cfg = {"configurable": {"thread_id": "project-7"}}
    app.invoke({"messages": [("user", "Recheck at 1.5x load.")]}, cfg)
    app.invoke({"messages": [("user", Q_revB)]}, cfg)
    # rev-B is sized for 0.75 k/ft. Nobody asked for that. Nothing raised.

# inspect what the agent will see next:
    print(app.get_state(cfg).values["messages"][-3:])
```

WHAT TO WATCH IN RUN A

No lookup_section call — it reads the section from State. Episodic memory as behaviour.

THE FAILURE, ON PURPOSE

Run C sizes rev-B for a load nobody specified. Completes; plausible; wrong.

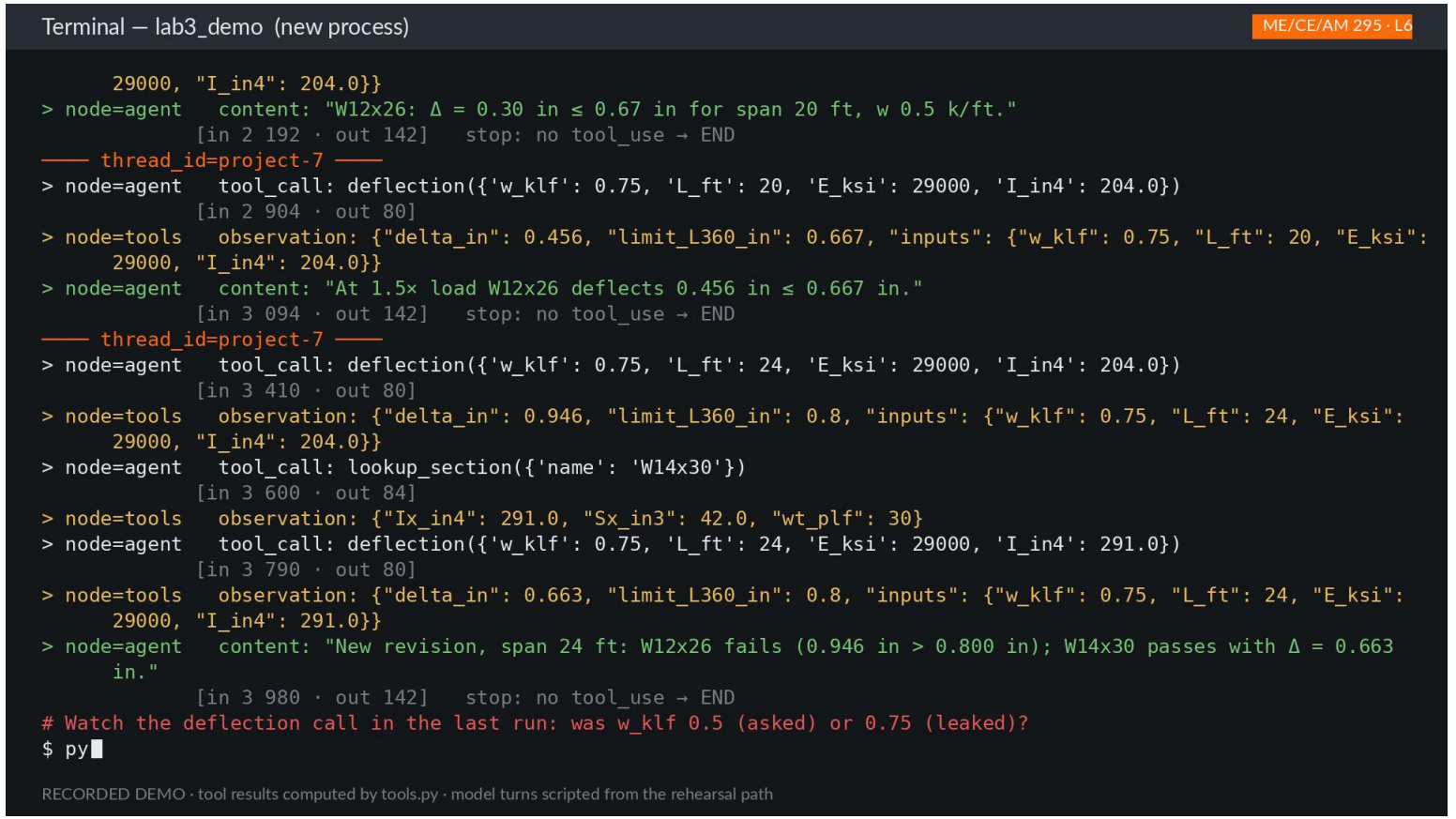
THE RULE FALLS OUT

One thread_id per revision. Constraints in State. get_state() shows what it will see next.

Three runs, one checkpoint file. Run A: resume. Run B: a fresh thread starts cold — correct. Run C: one project-wide thread, so the 1.5× load case is in State when rev-B starts — leakage, live.

Demo — Step 3, Resume and Leak

Run A remembers. Run B starts cold. Run C leaks — read w_klf in the last deflection call.



WHILE IT PLAYS

- Run A: no lookup — it remembers
- Run B: new thread, looks it up again
- Run C: w_klf = 0.75 — nobody asked
- get_state(): the 0.75 is in State
- Nothing raised

Recorded demo (63 s): tool results computed by tools.py; model turns follow the rehearsed path. Click to play. Scripts in lab3_demo/ on the course site.

From the trace alone — the printed stream and the checkpoint — which of the four silent failures could you detect, and which would need a second instrument?

Entrenchment · staleness · leakage · poisoning. For each: visible in the trace, visible only in get_state(), or invisible without a recomputation or a bound?

BRIDGE · FROM THE BUILD TO THE CHECKER

The agent now remembers and resumes. It still cannot tell whether it is right.

Wednesday's Rule 5: planning is the weak pillar. So give the plan a second reader — one that is not the planner.

PART 1
The Memory

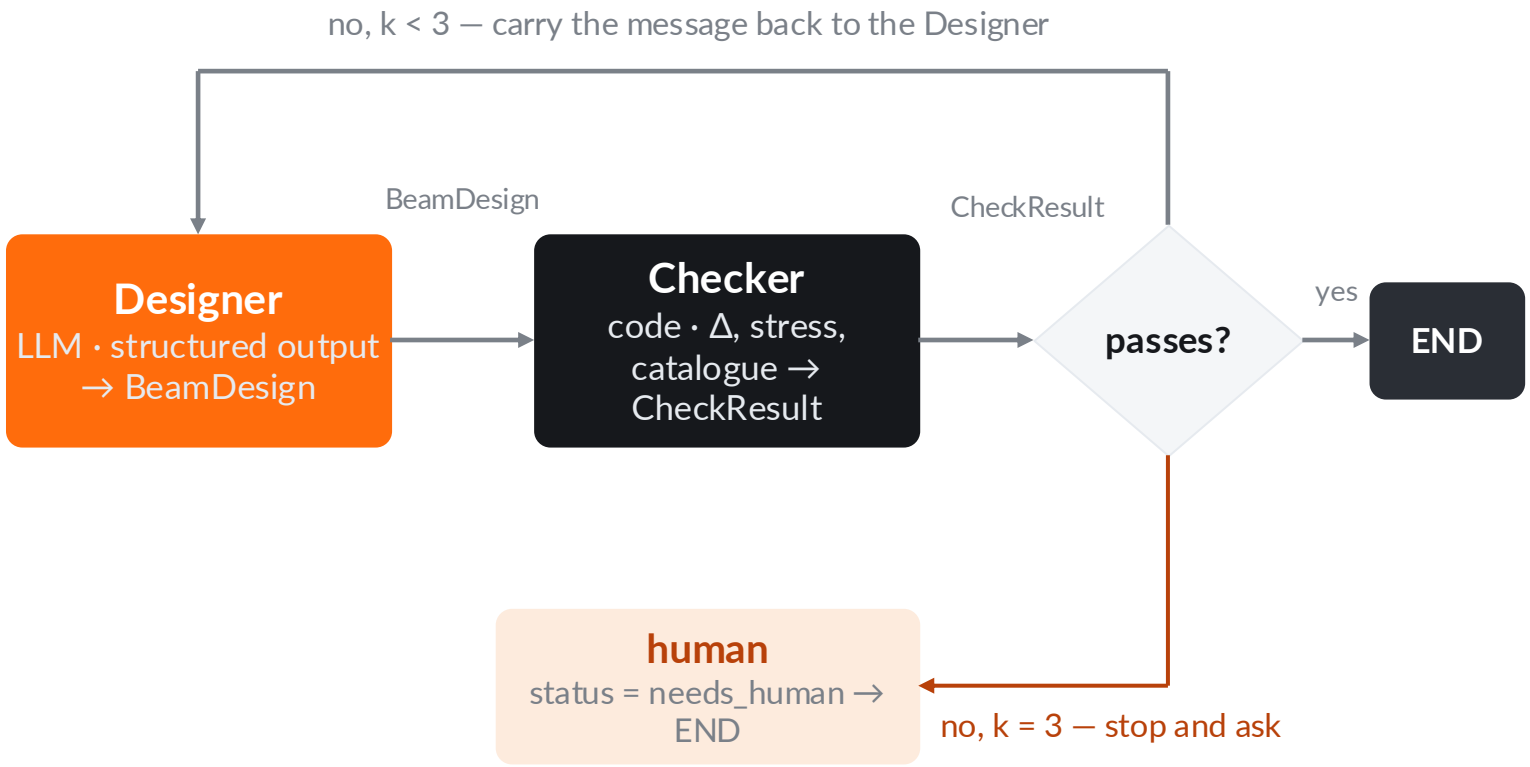
PART 2
The Build

PART 3
The Checker

why a second agent → the typed hand-off → a deterministic checker → the stop condition → a run, rev-A to rev-C

Designer / Checker — Not Its Own Evidence

A Designer proposes; a Checker — code — verdicts with a message; the loop stops on pass, or stops and asks.



THE CHECKER IS CODE

Deterministic where a formula exists: same input, same verdict, auditable. An LLM critic only where there is no formula.

THE MESSAGE IS THE SIGNAL

"Δ = 1.29 in > 0.67 in; need Ix ≥ 93 in⁴" is a signal. "Try again" is not.

THE STOP CONDITION IS A CAP

Pass, or k = 3 → stop and ask. Never "until it passes" — it will, for the wrong reason.

Anthropic, Building Effective Agents (2024): the evaluator–optimizer workflow. Swanson et al., Nature 2025: the Virtual Lab's Scientific Critic agent. The Checker here is a formula; Discussion Point 2 asks when it cannot be.

The Hand-off Is a Contract — Typed Both Ways

Lecture 4 made the output schema-valid. The Checker makes it physically valid — and its verdict is typed too.

```
class BeamDesign(BaseModel):                    # Designer -> Checker
    reasoning: str = Field(description="<= 60 words, first")
    section: str; span_ft: float; w_klf: float

class CheckResult(BaseModel):                   # Checker -> Designer
    passes: bool; delta_in: float; limit_in: float
    stress_ksi: float; message: str

def check_design(d: BeamDesign) -> CheckResult:   # no model here
    p = lookup_section(d.section)      # KeyError: not in catalogue
    L = d.span_ft * 12; w = d.w_klf / 12           # in, k/in
    delta = 5 * w * L**4 / (384 * 29000 * p["Ix_in4"])
    limit = L / 360
    stress = (d.w_klf * d.span_ft**2 / 8 * 12) / p["Sx_in3"]  # M/S
    ok = delta <= limit and stress <= 33.0
    msg = ("PASS" if ok else
           f"delta = {delta:.2f} in > {limit:.2f} in; "
           f"need Ix >= {p['Ix_in4'] * delta / limit:.0f} in4")
    return CheckResult(passes=ok, delta_in=delta, limit_in=limit,
                       stress_ksi=stress, message=msg)
```

WHY TYPED

Schema-valid ≠ physically valid. The schema is tested at the hand-off; the physics in the Checker.

WHY THE MESSAGE COMPUTES

It tells the Designer the Ix it must reach. An informative retry, not a coin flip.

WHY DETERMINISTIC

Same input, same verdict. No vote, no temperature, no faithfulness question. Evidence for Week 9.

Illustrative check: $\Delta = 5wL^4/384EI$ against L/360; bending stress M/S against 33 ksi (0.66 Fy, A992). Self-weight and load factors omitted for the demo; your capstone Checker carries the real code checks.

Designer / Checker in LangGraph — Two Nodes, One Edge

The counter lives in State; the conditional edge is the stop; the human is a node that ends the run.

```
class DCState(TypedDict):
    design: BeamDesign | None; result: CheckResult | None
    k: int; status: str

designer_llm = ChatAnthropic(model=MODEL, temperature=0) \
                   .with_structured_output(BeamDesign)
def designer(s):
    prompt = SPEC + (f"\nLast attempt failed: {s['result'].message}"
                     if s["result"] else "")   # last message only
    return {"design": designer_llm.invoke(prompt), "k": s["k"] + 1}
def checker(s):  return {"result": check_design(s["design"])}
def human(s):    return {"status": "needs_human"}  # ends, no retry
def route(s):
    if s["result"].passes: return END
    return "designer" if s["k"] < 3 else "human"    # the stop condition

g = StateGraph(DCState)
for n in (designer, checker, human): g.add_node(n.__name__, n)
g.add_edge(START, "designer"); g.add_edge("designer", "checker")
g.add_conditional_edges("checker", route); g.add_edge("human", END)
app = g.compile(checkpointer=memory)
out = app.invoke({"design": None, "result": None, "k": 0, "status": ""},
                 {"configurable": {"thread_id": "beam-rev-C"}})
```

ONE MESSAGE, NOT THE HISTORY

The Designer sees the spec and the last verdict only — the entrenchment cap from slide 5.

THE COUNTER LIVES IN STATE

k is checkpointed. A resumed run knows how many attempts it has spent.

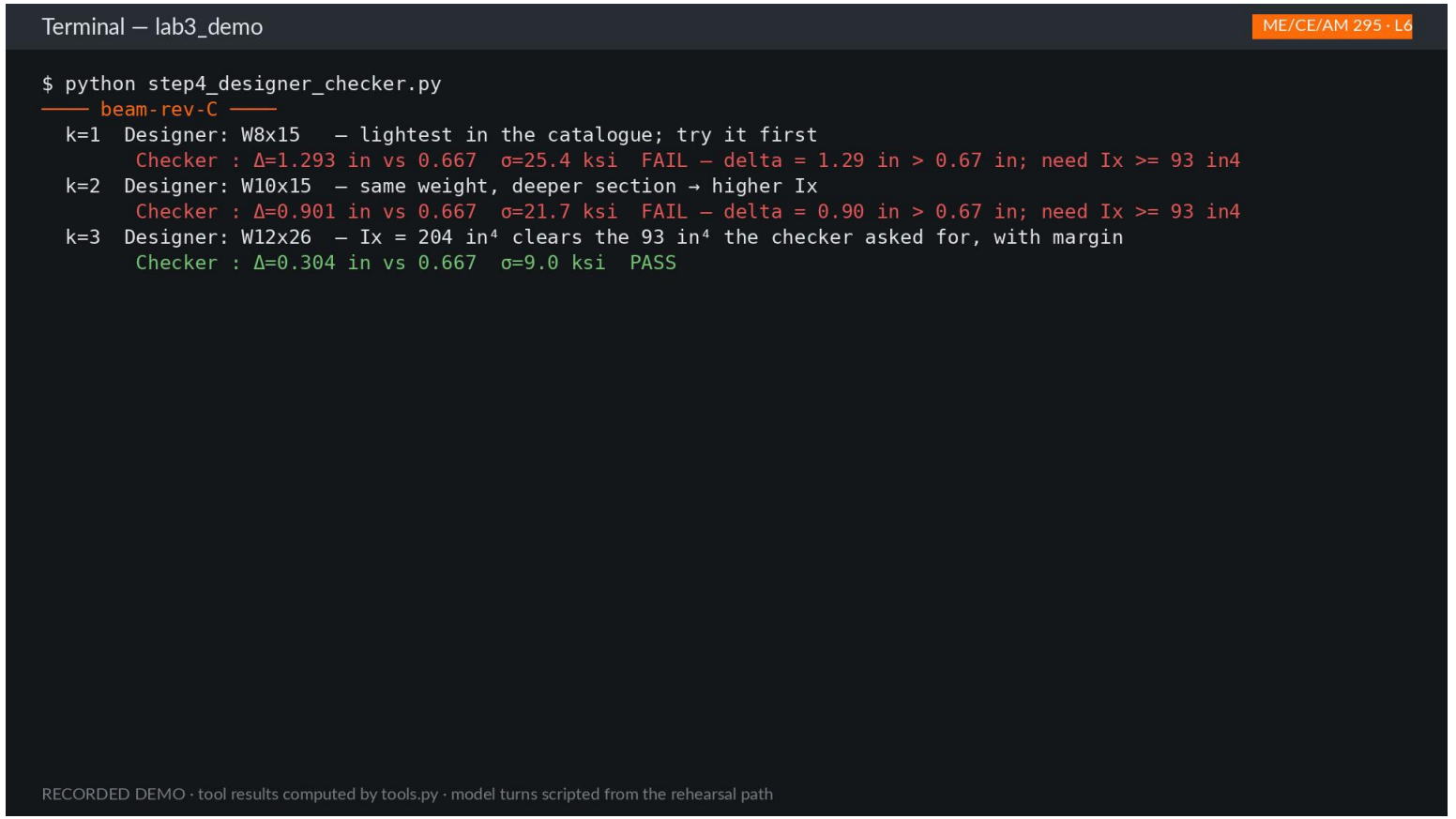
THE HUMAN IS A NODE

It writes needs_human and ends. Escalation is a state the graph reaches, not an exception someone may read.

with_structured_output returns a BeamDesign directly (Lecture 4's parse, framework-side). Verify against the installed langchain-anthropic. MODEL = the course model ID (pinned in the repo).

Demo — Designer / Checker, k = 1, 2, 3, then 20 runs

Three proposals, three verdicts, one pass. Then twenty runs, and the lightest section nobody chose.



WHILE IT PLAYS

- k=1 W8×15 fails; message says need Ix ≥ 93
- k=2 W10×15 under-corrects; fails
- k=3 W12×26 passes
- 20 runs: 0 escalations → “under ~15 %”
- Lightest that passes: W10×22

Recorded demo (31 s): tool results computed by tools.py; model turns follow the rehearsed path. Click to play. Scripts in lab3_demo/ on the course site.

What a Run Looks Like — rev-A to rev-C

Three proposals, three deterministic verdicts, one pass. Every row is in runs.db under beam-rev-C.

k	Designer proposed	Checker: Δ (in) vs 0.667	Stress (ksi) vs 33	Verdict	Message carried back
1	W8×15 — "lightest in the catalogue"	1.293	25.4	FAIL	Δ = 1.29 in > 0.67 in; need Ix ≥ 93 in^4
2	W10×15 — "deeper section, still light"	0.901	21.7	FAIL	Δ = 0.90 in > 0.67 in; need Ix ≥ 93 in^4 (W10×15 has 68.9)
3	W12×26 — "Ix = 204 clears 93 with margin"	0.304	9.0	PASS	— (END; checkpoint beam-rev-C closed)

LESSON 1 — k = 2 WOULD HAVE SHIPPED

Lighter, deeper, sounds right, fails. Without the Checker it ships — Rule 5, and nothing raises.

LESSON 2 — IT CHECKS WHAT YOU WROTE DOWN

W10×22 (Δ = 0.53 in, 22 plf) also passes and is lighter. Optimality was not in the contract. Write down what you care about.

Span 20 ft, w = 0.5 k/ft, E = 29 000 ksi, Δ_limit = L/360 = 0.667 in. Ix from AISC Table 1-1: W8×15 48.0 · W10×15 68.9 · W10×22 118 · W12×26 204 in^4. Stress = M/S with M = 25 k·ft.

Here the Checker is a formula. For the hardest check in your own capstone, is there one? If the Checker must be a model, what do you lose — and what did the Virtual Lab do about it?

Deterministic: same input, same verdict, auditable.

A model as Checker: a second opinion with its own faithfulness problem